

Basic Error Handling in Express.js

(Using Try–Catch and Custom Error Pages in an MVC-Based CRUD Application)

1. Overview

This tutorial explains how to implement **basic error handling** in an Express.js application that follows the **MVC pattern** and uses **MongoDB with Mongoose**. The existing project is a simple Contact Management CRUD system built in the previous sessions.

The tutorial covers:

- Why error handling is needed
- Two approaches to error handling in Express
- Validating MongoDB Object IDs
- Implementing **try–catch** blocks in controllers
- Creating and rendering **custom 404 and 500 error pages**
- Preventing Node/NodeMon from crashing due to unhandled errors
- Applying error handling across all CRUD controller methods

2. Error Handling Approaches in Express.js

2.1 Try–Catch (Basic Approach)

Used inside controller functions to catch synchronous and asynchronous logic errors.

2.2 Error Handling Middleware (Advanced Approach)

Uses `app.use((err, req, res, next) => {...})` and an extra parameter **next** to pass the error.

This tutorial focuses on **Try–Catch**.

3. Why Error Handling Is Needed

When the application receives:

- Invalid Contact IDs
- IDs that don't exist in the database
- Incorrectly formatted ObjectIDs
- Missing route parameters

Express and Mongoose can throw errors such as:

- `CastError`
- `ReferenceError`

- `MongoServerError`
- Internal Server Error

Without proper handling:

- The browser displays raw error pages
- NodeMon crashes
- The application stops responding

Thus, validating user inputs and wrapping controller logic in try-catch is essential.

4. Project Structure (MVC-Based)

```
project/  
|- index.js  
|- routes/  
|   └─ contactRoutes.js  
|- controllers/  
|   └─ contactController.js  
|- views/  
|   └─ 404.ejs  
|   └─ 500.ejs
```

All error handling will occur inside **controllers/contactController.js**.

5. Validating MongoDB Object IDs

Step 1: Import Mongoose in the Controller

```
import mongoose from "mongoose";
```

Step 2: Extract and Validate the Parameter

```
const paramId = mongoose.Types.ObjectId.isValid(req.params.id);
```

Step 3: If invalid, return a 404 error page

```
if (!paramId) {  
  return res.render("404", {  
    message: "Invalid ID"  
  });  
}
```

```
    });  
  }
```

Purpose: Ensures the ID is a **valid 24-character MongoDB ObjectId** before querying the database.

6. Creating Custom Error View Pages

6.1 Create `views/404.ejs`

Basic structure:

```
<%- include("partials/header") %>  
  
<h1>404 Error</h1>  
  
<% if(message) { %>  
  <h1><%= message %></h1>  
<% } else { %>  
  <h1>Page Not Found</h1>  
<% } %>  
  
<%- include("partials/footer") %>
```

6.2 Create `views/500.ejs`

```
<%- include("partials/header") %>  
  
<h1>500 Error</h1>  
  
<% if(message) { %>  
  <h1><%= message %></h1>  
<% } else { %>  
  <h1>Internal Server Error</h1>  
<% } %>  
  
<%- include("partials/footer") %>
```

These templates dynamically display custom error messages passed from the controller.

7. Implementing Try–Catch in Controller Functions

Below is a generic pattern applied across all CRUD methods.

7.1 Example: Fetching a Single Contact

Before (without error handling)

```
const contact = await Contact.findById(req.params.id);
res.render("contact", { contact });
```

After (with validation + try-catch)

```
export const getContact = async (req, res) => {

  // Validate ObjectId
  const paramId = mongoose.Types.ObjectId.isValid(req.params.id);
  if (!paramId) {
    return res.render("404", {
      message: "Invalid ID"
    });
  }

  try {
    const contact = await Contact.findById(req.params.id);

    // Check if ID exists in DB
    if (!contact) {
      return res.render("404", {
        message: "Contact Not Found"
      });
    }

    res.render("contact", { contact });

  } catch (error) {
    res.render("500", {
      message: error.message
    });
  }
};
```

What this achieves:

- Prevents malformed IDs from triggering CastError
- Catches database errors
- Ensures Node / NodeMon never crashes
- Displays correct error pages (404 or 500)

8. Applying Error Handling Across All CRUD Methods

8.1 Update Page (GET)

Validate ID → Try DB query → 404 if not found → Render page.

8.2 Update Action (POST)

Same pattern:

- Validate ObjectID
- Wrap DB update logic in try-catch
- If not found → show `404.ejs`
- If server error → show `500.ejs`

8.3 Delete Contact

```
try {
  await Contact.findByIdAndDelete(req.params.id);
} catch (error) {
  return res.render("500", { message: error.message });
}
```

8.4 Add New Contact (POST)

Even though no ID is involved:

- Wrap create logic in try-catch
- Render 500 error page if insertion fails

9. Applying Try-Catch to Home Page (List View)

```
export const home = async (req, res) => {
  try {
    const contacts = await Contact.find();
    res.render("home", { contacts });
  } catch (error) {
    res.render("500", { message: error.message });
  }
};
```

10. Difference Between `res.render()` and `res.redirect()`

`res.render("404")`

- Renders a template
- URL remains **unchanged**

`res.redirect("/404")`

- Changes the URL to `/404`
- Performs a browser redirect

Use redirect if you want the URL itself to reflect the error.

11. Final Outcomes of Implementing Error Handling

After adding these validations and catch blocks:

- No raw Mongoose/Node errors appear in browser
 - NodeMon no longer crashes due to unhandled promise rejections
 - Users receive clear messages:
 - Invalid ID
 - Contact Not Found
 - Internal Server Error
 - Application becomes stable and production-ready
-